Qwen3.8-Flash-Next

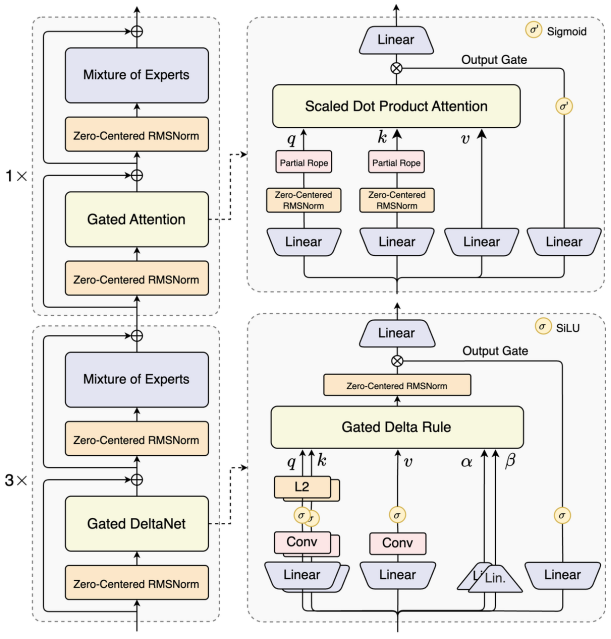
📌 论文：《On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability》
模型仓库：Qwen/Qwen3.8-Flash-Next
代码仓库：huggingface/transformers ·qwen4_exp
包含知识点：

- **GDN 混合注意力架构**
- **QSA（Qwen Sparse Attention）/ 两阶段蒸馏 CPT**
- **GR（Gated Residual）**，拓展阅读：mHC：Manifold-Constrained Hyper-Connections
- **N-gram Embedding**，拓展阅读：Engram：Conditional Memory via Scalable Lookup
- **Muon Optimizer**，拓展阅读：Muon 优化器

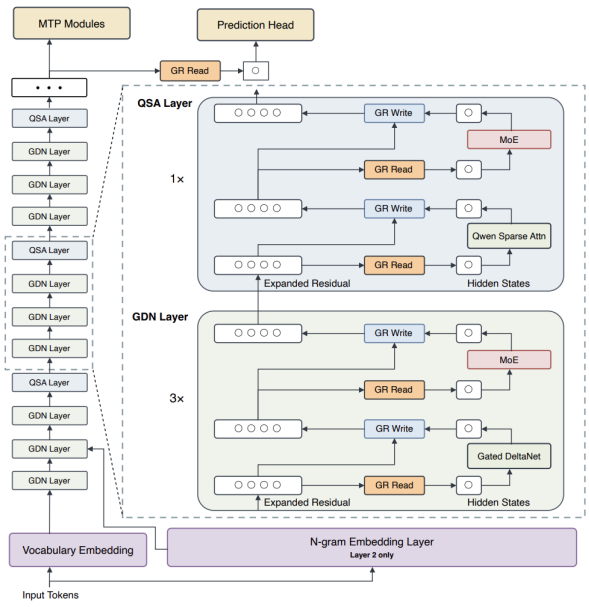
1. 模型架构

📌 **Qwen3.8-Flash-Next 相对上一代架构 Qwen-3.5 的改动：**

- **混合注意力架构**：使用交错层（interleaved）混合注意力，预训练阶段 Gated DeltaNet (GDN) 与 Full Attention 以 3:1 比例交错（48 层中 36 层为 GDN，另外 12 层保留全注意力）。线性层把前缀压缩进固定大小的状态，解决 KV cache 线性增长，全注意力层保留逐 token 检索能力。
- **CPT 阶段全注意力层换成 QSA**：继续预训练阶段（continued-pretraining，CPT）把 12 层全注意力替换为 QSA 稀疏注意力（Qwen Sparse Attention）。相比 DSA 的 token-level Top-K 打分，QSA 的索引器（indexer）先把序列连续 $r = 4$ 个 token 的 key 平均成 micro-block 再打分，把索引器自身 $O(n^2)$ 的开销降到 $O(n^2/r)$。
- **残差流替换为 GR（Gated Residual）**：残差流从恒等单流（ResNet）加宽到四条分支，读入时经过逐元素（element-wise）门控进行四条并行分支合并。
- **N-gram Embedding**：Embedding 采用 Engram 的 n-gram 查表（相比于传统的 unigram 查表），但是代价是表极大（Embedding 51B 参数量），但查表是确定性寻址、几乎不增加每个 token FLOPs，所以这张表留在主机内存里按需预取。



Qwen-3.5 ~ Qwen-3.8 架构



Qwen-3.8-next 架构

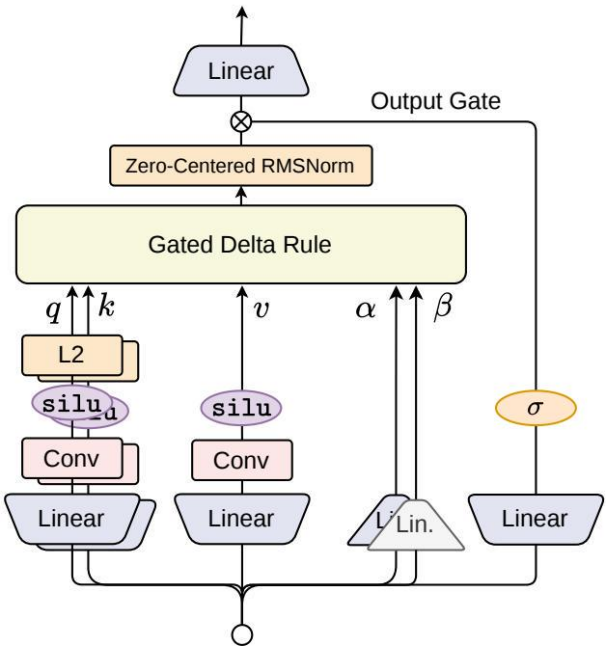
1.1 混合架构：GDN（Gated DeltaNet）+ 全注意力

动机： 全自注意力和局部注意力存在 trade-off。

- 全自注意力对每个历史 token 提供直接的、基于内容的访问，代价是计算平方增长、缓存线性增长。
- 滑动窗口注意力（SWA）把全局访问换成有界的局部感受野，降低了计算和缓存开销，但窗口之外的信息只能靠深度间接传播。

实现： 全局注意力和局部注意力交错排布，36 层线性注意力 GDN + 12 层全注意力。GDN 把前缀压缩进固定大小的循环状态并按当前内容更新，插在中间的全注意力层保留精确的 token 级检索。

本质上这个排布方式与 GLM-5.3-Flash、Kimi K3 的 3:1 混合是同一个逻辑：线性层的状态是固定大小的 $d_k \times d_v$ 矩阵，对序列长度不增长，代价是只能保留有损的历史摘要。全注意力保留完整 KV，代价是 $O(L)$ 的缓存。



1.1.1 Gated Delta Rule

线性注意力本质上是维护一个矩阵值循环状态 $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$。对每个 head，记 $\mathbf{q}_t, \mathbf{k}_t \in \mathbb{R}^{d_k}$、$\mathbf{v}_t \in \mathbb{R}^{d_v}$，GDN 维护状态 $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$，施加门控 delta 规则：

$$\widetilde{\mathbf{S}}_{t-1} = \alpha_t \mathbf{S}_{t-1}$$

$$\mathbf{e}_t = \mathbf{v}_t - \widetilde{\mathbf{S}}_{t-1}^{\top} \mathbf{k}_t$$

$$\mathbf{S}_t = \widetilde{\mathbf{S}}_{t-1} + \beta_t \mathbf{k}_t \mathbf{e}_t^{\top}$$

$$\mathbf{y}_t = \mathbf{S}_t^{\top} \mathbf{q}_t$$

- $\alpha_t \in (0,1)$ 是数据依赖的标量门，对整个状态矩阵施加统一衰减，作用等价于 L_2 正则（weight decay），每 head 一个标量（代码中`decay_t` 形状为 $[B, H, 1, 1]$）。
- $\beta_t \in (0,1)$ 是数据依赖的写入强度，控制本次键值对覆盖旧记忆的幅度，每 head 一个标量。
- $\mathbf{e}_t$ 是残差误差：用当前 key 读出已关联的值 $\widetilde{\mathbf{S}}_{t-1}^{\top}\mathbf{k}_t$，与真实的 $\mathbf{v}_t$ 求差。

等价地把上面的更新合成一步，得到标准更新表达式：

$$\mathbf{S}_t = \alpha_t (I - \beta_t \mathbf{k}_t \mathbf{k}_t^{\top}) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^{\top}$$

方法	遗忘门	delta rule	更新式（省略归一化）
Linear Attention	无	无	$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t\mathbf{v}_t^\top$
Mamba2	数据依赖标量 α_t	无	$\mathbf{S}_t = \alpha_t\mathbf{S}_{t-1} + \beta_t\mathbf{k}_t\mathbf{v}_t^\top$
GLA	逐通道 $\mathrm{Diag}(\boldsymbol{\alpha}_t)$	无	$\mathbf{S}_t = \mathrm{Diag}(\boldsymbol{\alpha}_t)\mathbf{S}_{t-1} + \mathbf{k}_t\mathbf{v}_t^\top$
DeltaNet	无	有	$\mathbf{S}_t = (I - \beta_t\mathbf{k}_t\mathbf{k}_t^\top)\mathbf{S}_{t-1} + \beta_t\mathbf{k}_t\mathbf{v}_t^\top$
GDN	数据依赖标量 α_t	有	$\mathbf{S}_t = (I - \beta_t\mathbf{k}_t\mathbf{k}_t^\top)\alpha_t\mathbf{S}_{t-1} + \beta_t\mathbf{k}_t\mathbf{v}_t^\top$
KDA	逐通道 $\mathrm{Diag}(\boldsymbol{\alpha}_t)$	有	$\mathbf{S}_t = (I - \beta_t\mathbf{k}_t\mathbf{k}_t^\top)\,\mathrm{Diag}(\boldsymbol{\alpha}_t)\mathbf{S}_{t-1} + \beta_t\mathbf{k}_t\mathbf{v}_t^\top$

上面的 4 个更新表达式可以对应到`torch_recurrent_gated_delta_rule`的解码路径：

> 公式和代码的对应关系：
> - α_t：`decay_t`
> - $\widetilde{\mathbf{S}}_{t-1}$：乘过`decay_t`之后的`last_recurrent_state`
> - $\widetilde{\mathbf{S}}_{t-1}^\top\mathbf{k}_t$：`kv_mem`
> - $\beta_t\mathbf{e}_t$：`delta`
> - $\mathbf{y}_t$：`core_attn_out[:, :, i]`

1.1.2 GDN 输入特征处理

内容特征（content features）的参数化用投影加短因果卷积计算：

$$\begin{aligned}\mathbf{q}_t &= \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_q\mathbf{x}_t))),\\ \mathbf{k}_t &= \mathrm{L2Norm}(\mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_k\mathbf{x}_t))),\\ \mathbf{v}_t &= \mathrm{SiLU}(\mathrm{ShortConv}(\mathbf{W}_v\mathbf{x}_t))\end{aligned}$$

写入强度与衰减的计算：

$$\beta_t = \sigma(\mathbf{W}_\beta\mathbf{x}_t), \quad \alpha_t = \exp[-\exp(\mathbf{A})\,\mathrm{softplus}(\mathbf{W}_\alpha\mathbf{x}_t + \mathbf{b}_\alpha)]$$

其中 $\mathbf{A}$ 是每 head 一个的可学习参数，保证 α_t 落在 $(0, 1)$。

1.1.3 GDN 输出门

逐 head 完成递推之后，输出经过归一化，再由一个输入相关的门调制得到输出：

$$\mathbf{o}_t = \mathbf{W}_o[\sigma(\mathbf{W}_z\mathbf{x}_t) \odot \mathrm{RMSNorm}(\mathbf{y}_t)]$$

1.1.4 全注意力保留 RoPE 位置编码

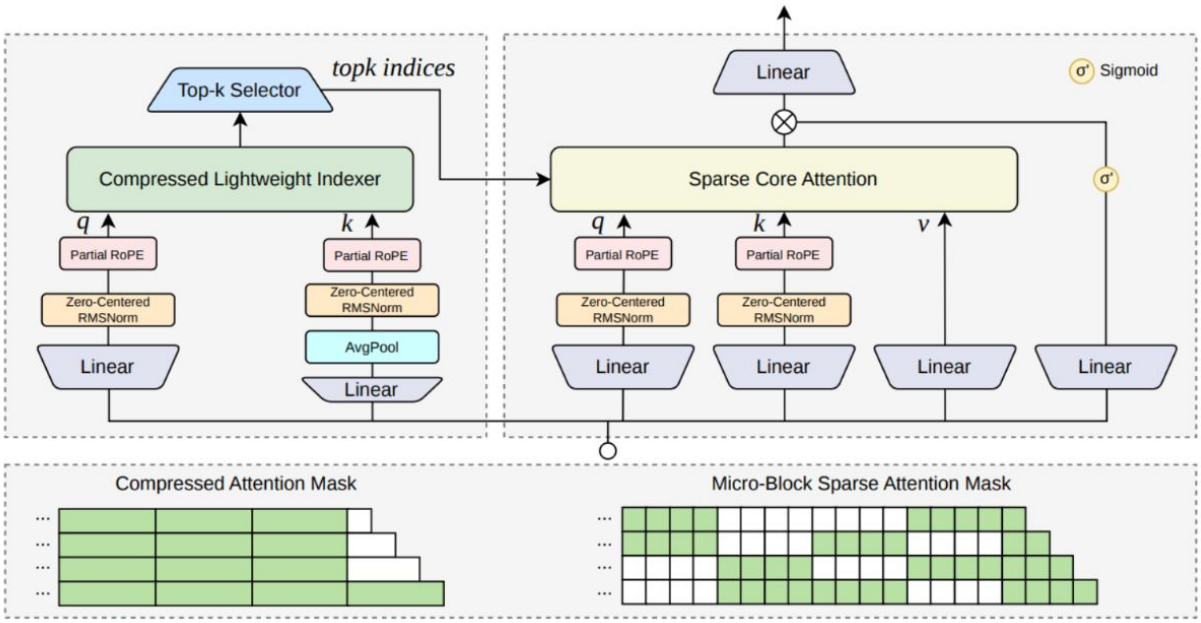
预训练阶段全注意力层保留 RoPE，理由是：RoPE 和不含位置编码的 NoPE 变体在预训练阶段表现差异不大，然而在后训练后，NoPE 变体会出现显著更高的无限生成现象，因此更容易导致生成无法停止。

> 论文原文：RoPE and a NoPE variant without positional encoding show little difference during pretraining, but the NoPE variant exhibits a substantially higher rate of endless generation after post-training and is therefore more likely to fail to terminate.

1.2 QSA（Qwen Sparse Attention）

动机： 稀疏注意力能缓解长上下文下 softmax 注意力的计算复杂度瓶颈。DSA （DeepSeek Sparse Attention）用轻量索引器（lightning indexer）生成 token-level 稀疏掩码，但随着序列变长，索引器自身$O(n^2)$的开销不可忽略。

实现： QSA 让索引器先把序列压缩成 micro-block 表示再打分，索引复杂度从 $O(n^2)$ 降到 $O(n^2/r)$。这与GLM-5.3-Flash的 IndexPool 是同一个思路：都是把细粒度的 token-level 打分改成 block-wise 打分。区别在压缩方式，IndexPool 用可学习权重的加权池化，QSA 用固定的平均池化。



1.2.1 压缩索引器（Compressed Lightweight Indexer）

索引器是 MQA 结构，H 个 query head 配一个共享 key head：

$$\widehat{\mathbf{q}}_i^h = \mathrm{RMSNorm}(\mathbf{W}_Q^h \mathbf{x}_i), \quad \mathbf{k}_i = \mathbf{W}_K \mathbf{x}_i$$

$\mathbf{W}_Q^h$ 是第 h 个 query 头的投影，$\mathbf{W}_K$ 是共享的 key 投影，$\mathbf{x}_i$ 是位置 i 的隐状态。

key 按 r 个 token 一组划成互不重叠的块并做平均池化压缩，记块 b 的起始位置为 $p_b = b \cdot r$，再加位置编码：

$$\widehat{\mathbf{k}}_b = \mathrm{RMSNorm}(\mathrm{AvgPool}(\mathbf{k}_{p_b:p_b+r-1})), \quad 0 \le b < \lfloor n/r \rfloor$$

$$\mathbf{q}_i^h = \mathrm{PRoPE}(\widehat{\mathbf{q}}_i^h, i), \quad \bar{\mathbf{k}}_b = \mathrm{PRoPE}(\widehat{\mathbf{k}}_b, p_b)$$

- r 是`indexer_compress_ratio`取 4。
- 位置编码用 partial RoPE，只作用在 128 维索引器头的其中 64 维。压缩发生在位置编码之前是为了避免把不同旋转相位的 token 表示平均到一起。

1.2.2 打分与选择（Scoring and Selection）

Block-wise 重要性分数按 ReLU 激活的 query-key 相似度在索引器头上求和：

$$I_{ib} = \begin{cases} \sum_{h=1}^{H} \mathrm{ReLU}(\langle \mathbf{q}_i^h, \bar{\mathbf{k}}_b \rangle), & p_b + r - 1 \le i \\ -\infty, & \text{otherwise} \end{cases}$$

主要注意条件是 **block-causal condition**：query 只能给已被完整观测的块打分。给定 token 预算 K，按块预算 $K_B = \lceil K/r \rceil$ 取 topK，选中的块展开回原始 token 下标，再与最后一个不完整块内的 token 合并（这部分保证被包含）：

$$\mathcal{B}_i = \mathrm{TopK}_{K_B}(\{I_{ib}\}_b), \quad \mathcal{S}_i = \mathrm{Expand}(\mathcal{B}_i) \cup \{r\lfloor (i+1)/r \rfloor, \ldots, i\}$$

> **QSA 索引器本质上做的事情：**先把每 4 个 token 的 key 平均成一个块表示，只给块打分、只对块取 top-k，再把选中的块展开回 token。打分量与排序量都因此降到原来的四分之一。

- $\mathbf{W}_Q^h$ 与 $\mathbf{W}_K$：融合成`index_qk_proj`，按`[n_heads*d, kv_heads*d]`切两段。
- $\widehat{\mathbf{q}}_i^h$：过完`q_layernorm`的q。$\mathbf{q}_i^h$：再过`apply_rotary_pos_emb`之后的q。
- $\mathbf{k}_i$：`raw_keys`。$\widehat{\mathbf{k}}_b$：`pooled_keys`（先`.mean`再`k_layernorm`）。
- $\bar{\mathbf{k}}_b$：`block_key_states`。p_b：`group_starts`，取每个块内第一个可见 token 的下标。
- I_{ib}：`scores`。$\mathcal{B}_i$：`selected_block_indices`。$\mathcal{S}_i$：`selected_tokens`。

1.2.3 两阶段蒸馏（Two-stage Distillation）

Qwen/Qwen3.8-Flash-Next 的 QSA 是在 CPT 阶段引入的，两个阶段都在 256K 序列长度的续训阶段进行。

- **Stage 1：稠密蒸馏（Dense Distillation）：**把主干的全序列注意力（full-sequence attention）分布蒸馏进索引器（indexer）。Token-level teacher 分布由核心注意力所有头的 softmax 分布求和再做 L_1 归一化，随后用 max pooling 对齐到块级：

$$\bar{a}_{ib} = \mathrm{MaxPool}(a_{i,p_b:p_b+r-1}), \quad \hat{\mathbf{a}}_i = \frac{\bar{\mathbf{a}}_i}{\|\bar{\mathbf{a}}_i\|_1},$$
$$\mathcal{L}_{\mathrm{KL}} = \frac{1}{N} \sum_i D_{\mathrm{KL}}(\hat{\mathbf{a}}_{i,:} \| \mathrm{Softmax}(\mathbf{I}_{i,:}))$$

> 用 max 而非 mean 聚合的理由是保留显著的 token-level 信号，否则会在聚合时被稀释。

第一阶段只训练索引器（indexer），步数 1000 步，学习率 1×10^{-3}，bs = 8，256K 序列，约 2B token。

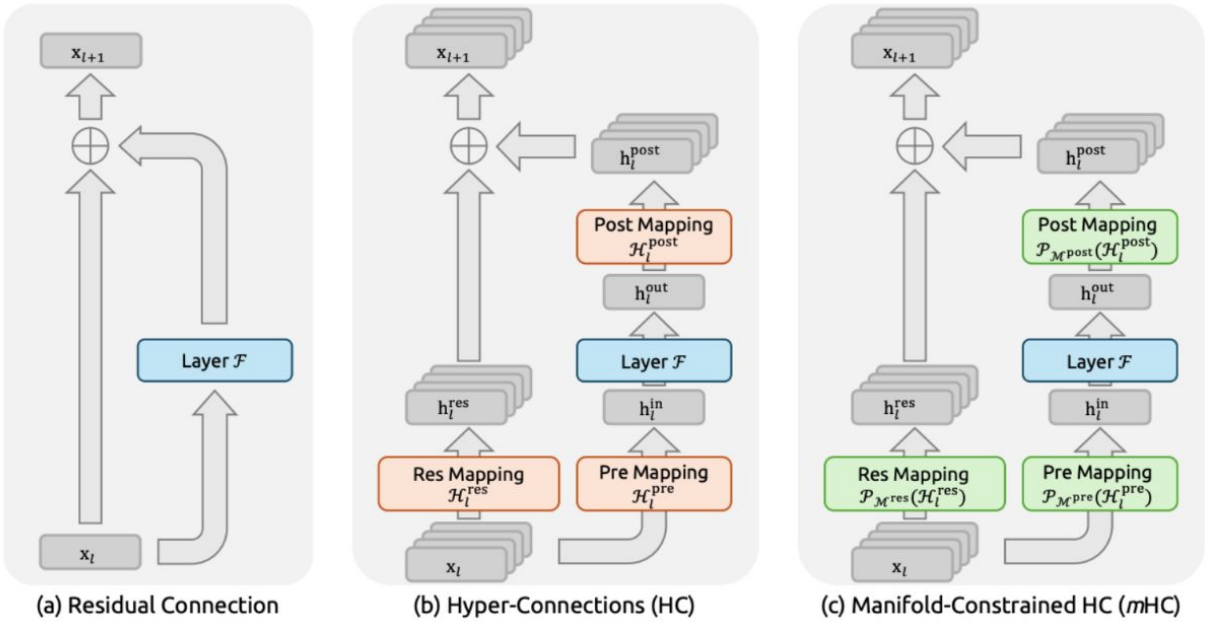
- **Stage 2：稀疏训练（Sparse Training）：**模型主干在索引器（indexer）指导下适应稀疏模式，KL 只在选中的 K_B 个块上计算，且 teacher 概率先在 $\mathcal{B}_i$ 内重归一化：

$$\mathcal{L}_{\mathrm{KL}} = \frac{1}{N} \sum_i D_{\mathrm{KL}}(\hat{\mathbf{a}}_{i,\mathcal{B}_i} \| \mathrm{Softmax}(\mathbf{I}_{i,\mathcal{B}_i}))$$

第二阶段联合训练 8000 步，学习率 2.5×10^{-5}，bs = 96，256K 序列，约 200B token。

1.3 GR（Gated Residual）：门控残差

> 前置阅读：mHC：Manifold-Constrained Hyper-Connections



(a) Residual Connection (b) Hyper-Connections (HC) (c) Manifold-Constrained HC (*m*HC)

按照mHC：Manifold-Constrained Hyper-Connections的逻辑，设 $\mathbf{x}_l \in \mathbb{R}^{n\times C}$ 是第 l 层的 n 条残差流，残差流从 1 条加宽到 n 条，就有三个操作必须阐明：怎么读（对应映射 $\mathcal{H}_l^{\text{pre}}$）、怎么写（对应映射 $\mathcal{H}_l^{\text{post}}$）、流与流之间混合（对应映射 $\mathcal{H}_l^{\text{res}}$）。HC、mHC、GR 本质区别就在这三个操作的设计上。

1.3.1 回顾 HC 与 mHC

标准残差只有单流，等价于恒等读、恒等写、恒等混合：

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l, \mathbf{W}_l)$$

HC 把流加宽成 n 条，并把上面三个操作设置分别为三个可学习映射算子：

$$\mathbf{x}_{l+1} = \mathcal{H}_l^{\text{res}}\mathbf{x}_l + \mathcal{H}_l^{\text{post}\top}\mathcal{F}(\mathcal{H}_l^{\text{pre}}\mathbf{x}_l, \mathbf{W}_l)$$

- $\mathcal{H}_l^{\text{pre}} \in \mathbb{R}^{1\times n}$ 是**读操作**：n 个标量，把 n 条流加权成 1 条 C 维向量送进子层 $\mathcal{F}$。
- $\mathcal{H}_l^{\text{post}} \in \mathbb{R}^{1\times n}$ 是**写操作**：n 个标量，把子层的 1 条输出展开写回 n 条流。
- $\mathcal{H}_l^{\text{res}} \in \mathbb{R}^{n\times n}$ 是**混合操作**：让第 i 条流的新值含第 j 条流的成分。

把上面的 HC 表达式递推展开从第 l 层到第 L 层：

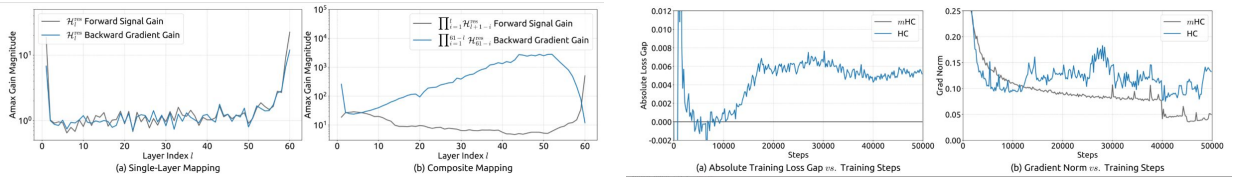
$$\begin{aligned}\mathbf{x}_L = &\left(\prod_{i=1}^{L-l}\mathcal{H}_{L-i}^{\text{res}}\right)\mathbf{x}_l \\ &+\sum_{i=l}^{L-1}\left(\prod_{j=1}^{L-1-i}\mathcal{H}_{L-j}^{\text{res}}\right)\mathcal{H}_i^{\text{post}\top}\mathcal{F}(\mathcal{H}_i^{\text{pre}}\mathbf{x}_i, \mathbf{W}_i)\end{aligned}$$

其中 $\prod_{i=1}^{L-l}\mathcal{H}_{L-i}^{\text{res}}$ 是 residual stream 的复合残差映射（composite mapping）

问题在于：$\mathcal{H}_l^{\text{res}}$是无约束可学习矩阵，多层连乘$\prod_{i=1}^{L-l}\mathcal{H}_{L-i}^{\text{res}}$极容易发生梯度爆炸或消失。或者说复合映射是无约束的，在层数增加后会愈发偏离 identity mapping。论文里用 Amax Gain Magnitude 量化了这一过程：

- Forward：复合映射的最大行绝对和（最大行和），衡量前向传播的信号放大倍数。
- Backward：复合映射的最大列绝对和（最大列和），衡量反向传播的梯度放大倍数。

通过 27B 模型的训练消融，可以得出：单层映射 $\mathcal{H}_l^{\text{res}}$ 的 Amax Gain Magnitude 理想情况接近 1。复合映射 $\prod_{i=1}^{L-l}\mathcal{H}_{L-i}^{\text{res}}$ 的峰值接近 3000»1，直接导致了训练不稳定。对应的，训练在 12k step 左右出现明显的 loss spike 和 gradient norm 爆炸：



mHC 的解法是保留 $\mathcal{H}_l^{\text{res}}$、把它约束到双随机矩阵流形上（每行和、每列和都为 1，元素非负），用 20 轮 Sinkhorn-Knopp 迭代投影。双随机矩阵谱范数不超过 1 且对乘法封闭，连乘项因此不再放大信号。代价是每层多 20 轮迭代，而且残差流宽 n 倍带来的访存开销并没有解决。

1.3.2 GR 的解法：去除多流混合

Qwen-3.8-Next 通过消融认为 $\mathcal{H}_l^{\text{res}}$ 带来的增益有限：$\mathcal{H}^{\text{res}}$ adds little. Once the read and the write are expressive enough, adding the $n_r \times n_r$ mixing operator brings no significant improvement.

因此 GR 直接去掉混合算子，更新式：

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{H}_l^{\text{post}\top}\mathcal{F}\left(\frac{1}{n}\sum_{i=1}^{n}\mathcal{H}_{l,i}^{\text{pre}} \odot \widetilde{\mathbf{x}}_{l,i},\ \mathbf{W}_l\right)$$

对比以上 HC 的更新式的区别：

1. 残差项从 $\mathcal{H}_l^{\text{res}}\mathbf{x}_l$ 退化成 $\mathbf{x}_l$，可以看作 $\mathcal{H}_l^{\text{res}}$ 被单位矩阵取代。因此去除了递推展开连乘项，也就不需要 mHC 的双随机流形与 Sinkhorn 迭代。
2. 读从矩阵乘变成逐元素乘 $\odot$，因为 $\mathcal{H}_l^{\text{pre}}$ 的形状从 $1 \times n$ 拓展为 $n \times C$，同时 $\widetilde{\mathbf{x}}_l$ 为原始的 $\mathbf{x}_l$ 的归一化。

完整的 GR 计算流程：

1. 先把 n 条流独立归一化：

$$\widetilde{\mathbf{x}}_{l,i} = \text{RMSNorm}(\mathbf{x}_{l,i};\boldsymbol{\gamma}_i), \quad i = 1,\ldots,n$$

$\mathbf{x}_{l,i} \in \mathbb{R}^C$ 是第 i 条流，$\boldsymbol{\gamma}_i \in \mathbb{R}^C$ 是第 i 条流独立的可学习缩放。

> 与 mHC 的区别：mHC 是先 $\text{vec}(\mathbf{x}_l)$ 拼成 $1 \times nC$ 再整体做一次 RMSNorm。GR 是 n 条流独立归一化，各自维护可学习缩放 $\boldsymbol{\gamma}_i$。

2. 从拼接 $\text{vec}(\widetilde{\mathbf{x}}_l) \in \mathbb{R}^{nC}$ 计算出读、写变换：

$$\mathcal{H}_l^{\text{pre}} = \text{unvec}\,\sigma\left(\mathbf{W}_u\,\text{SiLU}\left(\frac{1}{n}\mathbf{W}_d\,\text{vec}(\widetilde{\mathbf{x}}_l)\right)\right) \in \mathbb{R}^{n\times C}$$

$$\mathcal{H}_l^{\text{post}} = 2\sigma\left(\frac{1}{n}\mathbf{W}_w\,\text{vec}(\widetilde{\mathbf{x}}_l)\right) \in \mathbb{R}^{n}$$

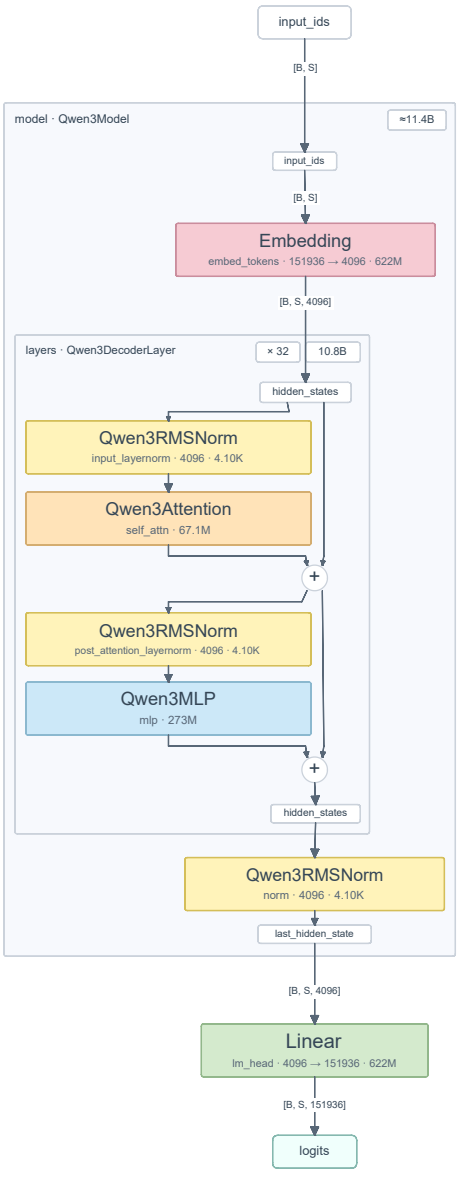
- $\mathbf{W}_d \in \mathbb{R}^{r\times nC}$ 把 $nC = 4 \times 2560 = 10240$ 维压到 low-rank 维度 r，$\mathbf{W}_u \in \mathbb{R}^{nC\times r}$ 再升回 10240 维。low-rank 维度 $r = C/8 = 320$，即配置里的`hc_lowrank`，目的是压缩参数量（相比 10240^2 满秩矩阵变换）。
- $\mathbf{W}_w \in \mathbb{R}^{n\times nC}$ 即 $[4, 10240]$，计算 4 个分流系数。
- σ 是 Sigmoid，unvec 把 10240 维 reshape 为 $n \times C$。

3. 使用更新式进行计算：$\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{H}_l^{\text{post}\top} \mathcal{F}\left(\frac{1}{n}\sum_{i=1}^{n} \mathcal{H}_{l,i}^{\text{pre}} \odot \widetilde{\mathbf{x}}_{l,i},\ \mathbf{W}_l\right)$

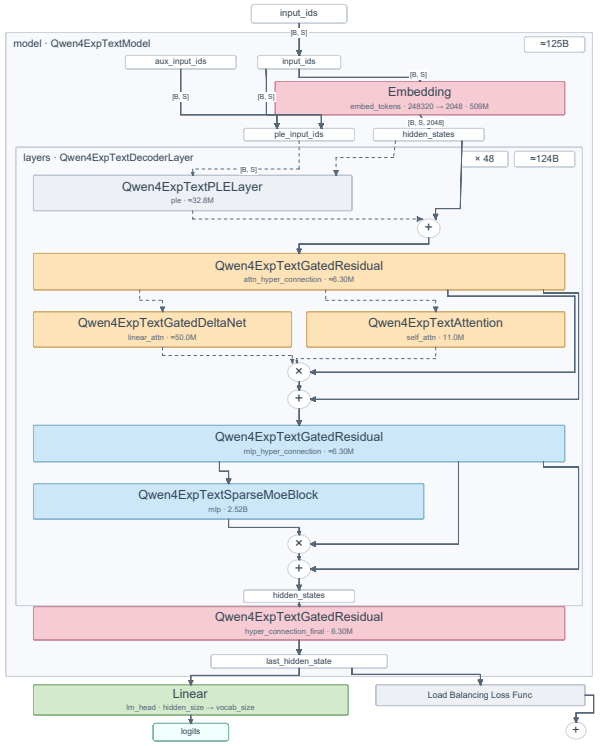
> **在三个操作上，GR 与 mHC 对比：**
>
> - $\mathcal{H}_l^{\text{pre}}$：mHC 是 $\sigma(\cdot) \in \mathbb{R}^{1\times n}$，$n$ 个权重标量。GR 是 $\sigma(\cdot) \in \mathbb{R}^{n\times C}$，$n \times C$ 个门控标量。
> - $\mathcal{H}_l^{\text{post}}$：两边都是 $2\sigma(\cdot)$，形式一致。
> - $\mathcal{H}_l^{\text{res}}$：mHC 是 Sinkhorn$(\cdot) \in \mathbb{R}^{n\times n}$。GR 去除这个操作（恒等变换）。
>
> 以上操作对应 GR 的消融结论，读的粒度比写的粒度重要，同样的细化施加在写上几乎没有收益，因此施加逐元素（element-wise）的门控（gating）。代码实现逻辑参考以下代码：

- $\mathbf{x}_l$：`hyper_input`，$\widetilde{\mathbf{x}}_l$：`hyper_input_normed`。
- γ_i：`hc_norm.weight` 的第 i 段。
- $\mathbf{W}_d$：`input_mix_weight_down`，$\mathbf{W}_u$：`input_mix_weight_up`，$\mathbf{W}_w$ 对应`block_inject_weight`。
- $\mathcal{H}_l^{\text{pre}}$：`unflatten` 之后的`input_mix_weight`，$\mathcal{H}_l^{\text{post}}$：`injection_weights`。

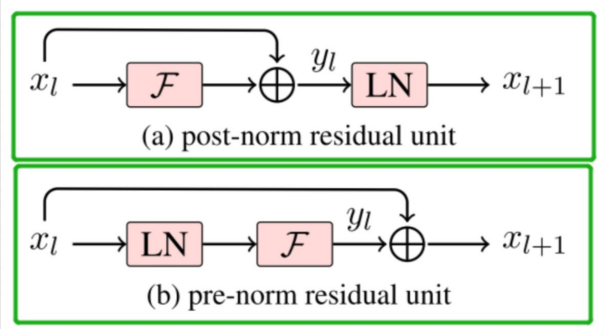
1.3.3 GatedNorm 替代 Pre-Norm



Qwen-3 Pre-Norm 示例



Qwen-3.8-Next GatedNorm 替代 Pre-Norm



Pre-Norm\Post-Norm 对比

在前序工作《A Unified View of Attention and Residual Sinks: Outlier-Driven Rescaling is Essential for Transformer Training》中有结论：

在 RMSNorm 之后加一个轻量的逐元素门可显著改善训练稳定性，即 GatedNorm，公式表达为：

$$\mathrm{GatedNorm}(\mathbf{u}) = \mathrm{RMSNorm}(\mathbf{u}) \odot \sigma(\mathbf{W}_2\,\mathrm{SiLU}(\mathbf{W}_1\,\mathrm{RMSNorm}(\mathbf{u})))$$

其中 $\mathbf{W}_1$ 与 $\mathbf{W}_2$ 是一对低秩投影，先降维再升回原维度。

把这个表达式与上面 GR 的 $\mathcal{H}_l^{\mathrm{pre}}$ 更新式对比，本质上两者是同一个结构：归一化、低秩降维、SiLU、升维、Sigmoid、再逐元素乘，区别仅在于 GatedNorm 作用于单流，GR 的读作用于 n 条流拼起来的向量。

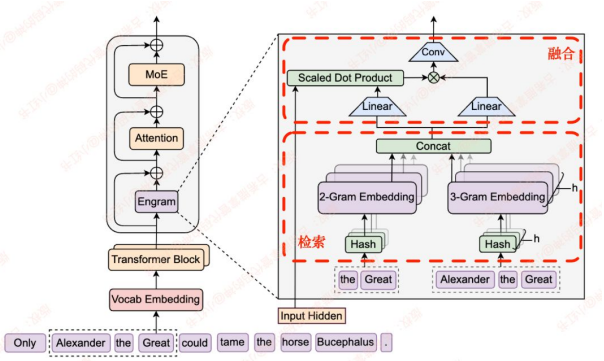
因此 Qwen-3.8-Next 将两者合并成一个算子：GR 的读已经包含归一化与门控，所以它取代了子层的预归一化（Pre-Norm），参考上图可以清晰看出这个结构。

还有一个需要注意的细节是 n（n=4）条流的进出口（如何从单流进行拓展），N 流的入口状态来自`repeat`而不是 n 组独立初始化：

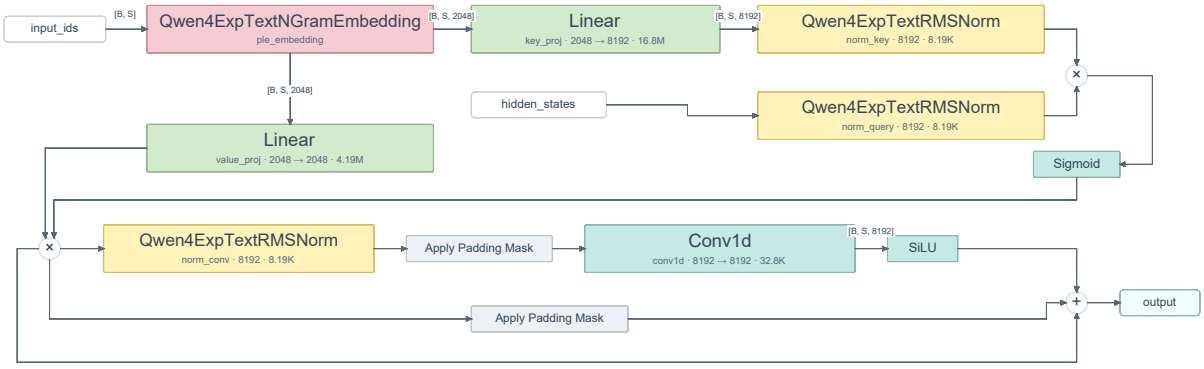
1.4 N-gram Embedding

> 前置阅读 Engram：Conditional Memory via Scalable Lookup：通过 Embedding 进行模型的记忆扩容。

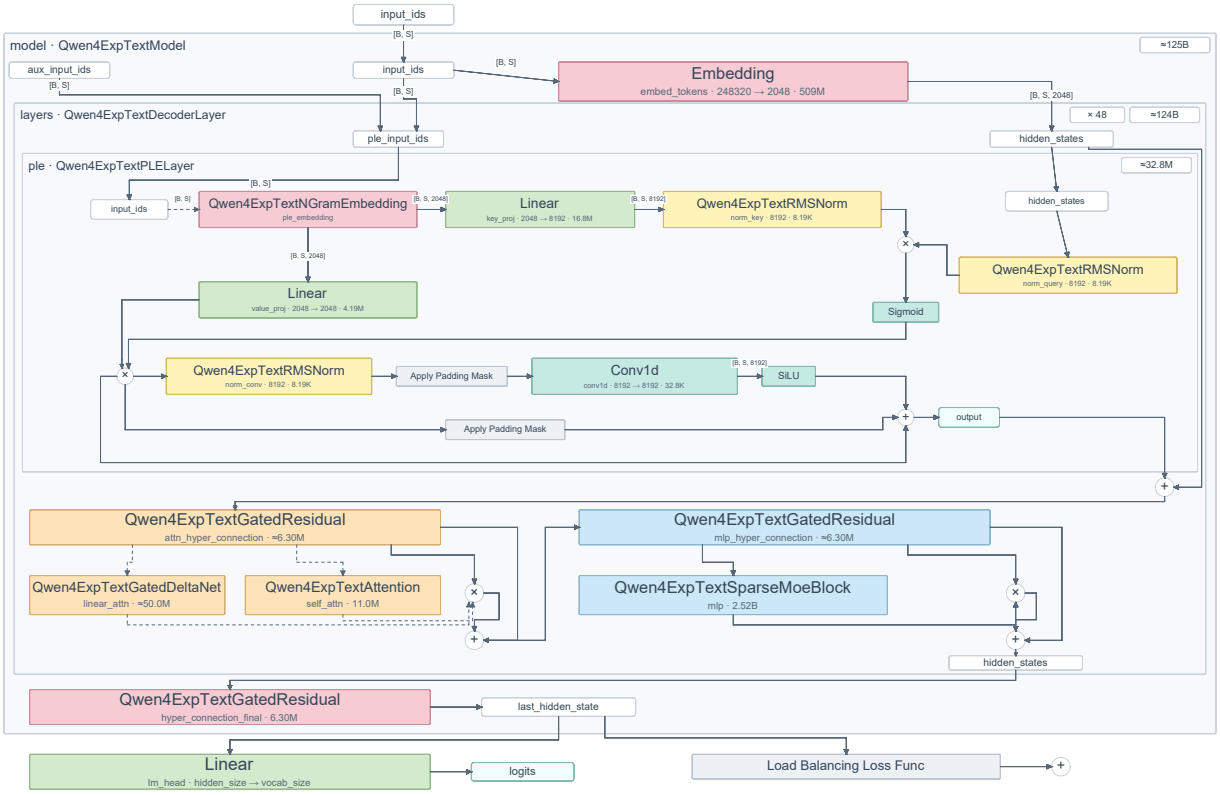
N-gram Embedding 把当前 token id（uni-gram）为 key 查表推广为以局部上下文（n-gram）为查表的 key：以每个 token 结尾的 n-gram 作为 Key 查表，取回的向量增强该 token 的表示。这个做法的优点是在几乎不增加逐 token FLOPs 的前提下进行记忆扩容，并且使用主机内存卸载与异步预取进行确定性寻址。



Engram 架构示意图



Qwen4ExpTextPLELayer



Qwen4ExpTextPLELayer 在 Qwen-3.8-next 的位置

从宏观上看 N-gram Embedding 模块就是 Engram：主体逻辑都是用后缀 n-gram 做 key、多头哈希查表、再与当前隐状态门控融合。

Engram：在标准的 Transformer 主干结构上增加 Engram 条件记忆模块，通过残差连接将 Engram 串联到 Transformer-Block 主干网络：$\mathbf{H}^{(\ell)} \leftarrow \mathbf{H}^{(\ell)} + \mathbf{Y}$，注意模型结构中只有部分 Transformer-Block 增加 Engram 模块。

增加 Engram 模块后，在架构上静态模式存储（Engram）与动态计算（Attention、FFN）分离。

新的模型结构处理每一个位置t的 token 方法分为两个阶段：给定一个输入序列 $X = (x_1, \ldots, x_T)$ 以及第 ℓ 层的隐藏状态 $\mathbf{H}^{(\ell)} \in \mathbb{R}^{T \times d}$，Engram 模块通过检索和融合两个阶段处理每个位置 t 的特征：

- **检索：**提取并压缩后缀 N-gram，通过多头哈希检索静态嵌入向量（记忆向量）。
- **融合：**检索到的静态嵌入向量与当前隐藏状态进行 CrossAttn 动态融合，并通过一个轻量卷积扩展表达能力。

我们直接复用 Engram：Conditional Memory via Scalable Lookup 中的符号表示：$g_{t,n}$ 是位置 t 结尾的 n-gram，$\phi_{n,k}$ 是第 n 阶第 k 个头的哈希函数，$\mathbf{E}_{n,k}$ 是对应的嵌入表，$\mathbf{e}_t$ 是拼起来的记忆向量。

检索阶段要解决的问题：这步需要用 n-gram 组合进行记忆向量召回，但是直接参数化所有可能的 n-gram 组合（如 3-gram）导致参数爆炸。

解决方法：

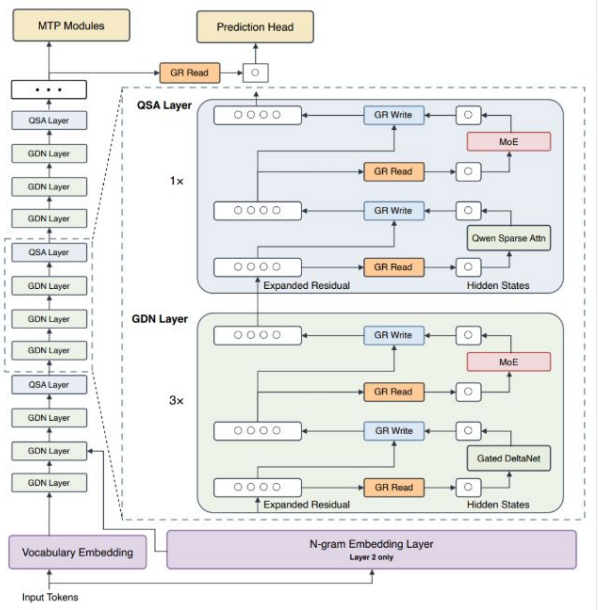
1. **哈希降维（解决组合参数爆炸问题）**：对 n-gram token ids 序列 $g_{t,n}$，通过哈希函数 $\phi_{n,k}$ 映射到固定维度的索引 $z_{t,n,k}$，从嵌入表 $\mathbf{E}_{n,k}$ 中检索向量 $\mathbf{e}_{t,n,k}$：
   - 计算 n-gram 的哈希值：$z_{t,n,k} \triangleq \phi_{n,k}(g_{t,n})$
   - 在嵌入表（记忆）中检索：$\mathbf{e}_{t,n,k} = \mathbf{E}_{n,k}[z_{t,n,k}]$
   - 不同 n-gram 的记忆向量进行 concat：$\mathbf{e}_t \triangleq \|_{n=2}^{N} \|_{k=1}^{K} \mathbf{e}_{t,n,k}$

   > $g_{t,n}$ 是以位置 t 结尾、长度为 n 的 token 序列（n-gram）。$N = 3$，所以 n 取 2 和 3 两阶。
   > $K = 8$ 是每一阶的 head 数，总头数 $(N-1) \times K = 16$。
   > $\phi_{n,k}$ 是哈希函数：各位置 token id 乘上该位置的乘子后逐位异或（multiplicative-XOR），与 Engram 相同。
   > $\mathbf{e}_{t,n,k} \in \mathbb{R}^{160}$，16 个 head 拼接起来 $\mathbf{e}_t \in \mathbb{R}^{2560}$，等于主干网络维度 C。

2. **多头机制（解决哈希冲突问题）**：对每个 n-gram 阶数 n，使用 k 个独立哈希头（每个头使用相同的哈希函数和不同嵌入表），降低哈希碰撞的影响。

相比 Engram：Conditional Memory via Scalable Lookup，Qwen-3.8-Next N-gram Embedding 实现上的区别：

- $g_{t,n}$ 直接用原始 token id，不做归一化压缩，相比之下 Engram 需要做分词器压缩（Tokenizer Compression）。
- 每个头的表长 $|\mathbf{E}_{n,k}|$ 取各自不同的质数，降低两个 ngram 不会在 8 个头上 hash 碰撞。
- Qwen 只在第二层上使用 N-gram Embedding（`ple_layer_ids: [2]`）。选第 2 层的理由不在表征上，主要是考虑用第一层的计算时间掩盖主机内存的预取延迟。论文：`allowing host-memory prefetching to overlap with the computation of the first layer`。
- 51B 的 Embedding Table 放在 host 内存。$z_{t,n,k}$ 只由输入 token 算出、与模型状态无关，下一步要查哪几行可以提前算出来，于是 $\mathbf{E}_{n,k}$ 可以留在主机内存里异步预取。



Qwen-3.8-next 架构

还有一个需要关注的区别：Engram 的融合是在单条残差流上加：$\mathbf{H}^{(\ell)} \leftarrow \mathbf{H}^{(\ell)} + \mathbf{Y}$。Qwen 因为 GR 把残差流加宽成了 n 条，因此需要融合方法端改动：key 投到 nC 维、n 条流各一份。value 只有 C 维、n 条流共用。

解释：每条流各自判断是否写入这条记忆，写入的内容却是同一份。相比之下 Engram 是单流，不存在这个区分。

		Knowledge			STEM		Reasoning	Chinese		Multilingual
Vocab. Scale	Loss	MMLU	MMLU-Pro	SuperGPQA	MATH	GSM8K	BBH	C-Eval	CMMLU	MMMLU
None	1.585	62.78	33.43	20.97	32.52	59.21	53.40	66.91	68.10	54.06
20×	1.553	64.14	34.46	21.83	**37.38**	**65.09**	57.13	71.75	72.29	55.94
50×	1.541	64.71	35.80	21.49	37.32	64.00	**57.56**	72.12	72.48	56.64
100×	1.534	64.70	**35.87**	**21.93**	36.98	63.08	56.03	73.75	72.73	**56.65**
200×	**1.526**	**64.85**	35.21	21.11	35.34	62.96	56.23	**74.94**	**73.24**	55.82

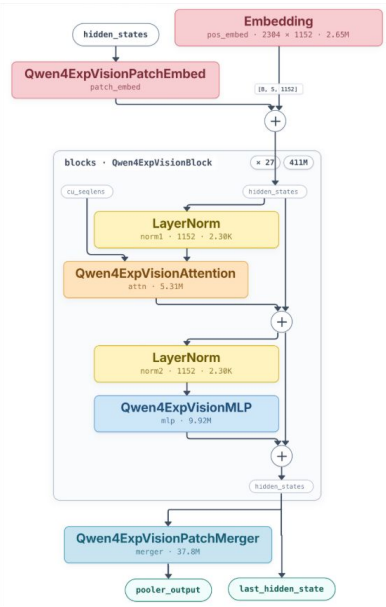
固定预算、把 n-gram 词表从 20 倍缩放到 200 倍基础词表大小：loss 随词表单调下降，下游任务表现不一定变好，MATH 在 20 倍就到达最优，BBH 在 50 倍到达最优，只有中文的 C-Eval 与 CMMLU 随词表一致上升。这个在Engram：Conditional Memory via Scalable Lookup实验中也有体现。

1.5 Vision Encoder

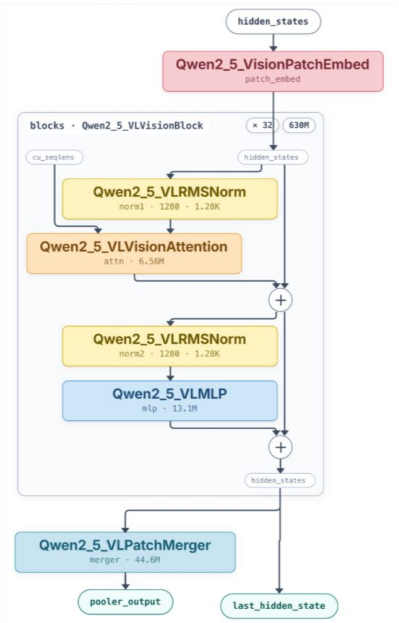
1.5.1 Positional Embedding

Qwen3-VL以来的 Qwen 视觉语言模型（VLM），在 Vision Encoder 同时有绝对位置编码、2D RoPE 两套位置编码生效：

- 绝对位置只在作用在进入第一层之前。绝对位置表给出的是一张全局的、随分辨率插值伸缩的坐标图，只作用于输入。本质就是最原始的ViT 的位置编码。
- 2D RoPE 则作用在每一个 ViT 层的注意力。RoPE 给出的是每一层注意力内部的相对位移，只作用于 q 与 k 的内积。2D-RoPE 不再重复叙述，具体可以参考：2D-RoPE。



Qwen3.8 Vision Model

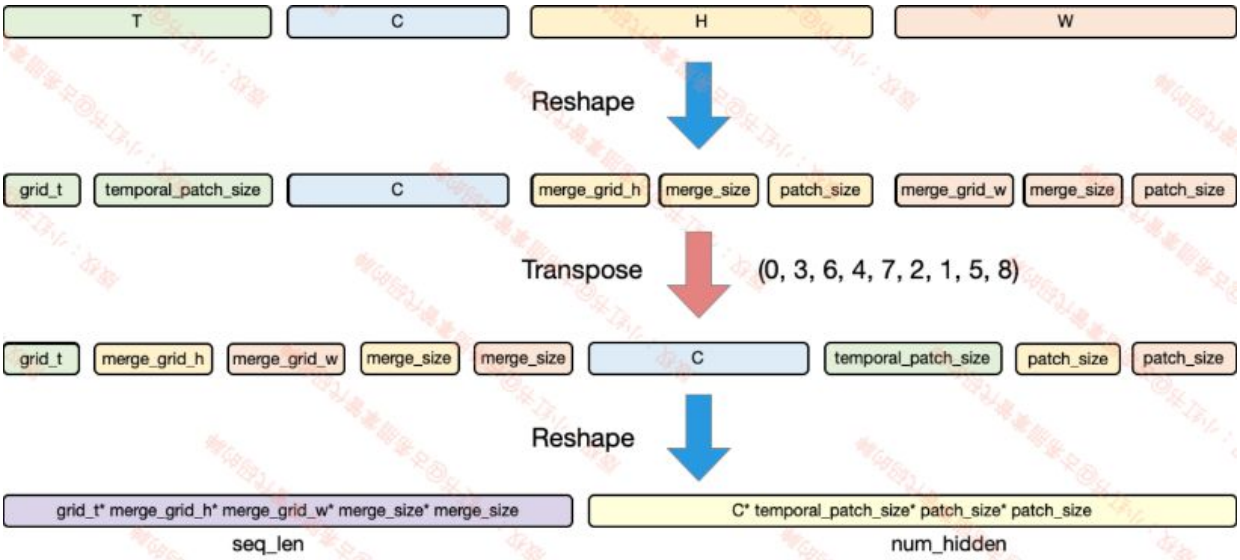


Qwen2.5 Vision Model

绝对位置编码`nn.Embedding(2304, 1152)`本质是训练阶段可学习的 Embedding 表，$2304 = 48^2$ 对应固定的 48×48 网格。对任意分辨率输入，先用双线性插值把网格重采样到实际的 patch 网格上，再与 patch 嵌入相加：

1.5.2 PatchMerger

`Qwen4ExpVisionPatchMerger` 把 2×2 的空间邻域拼成一个 token 再投影到主干宽度：

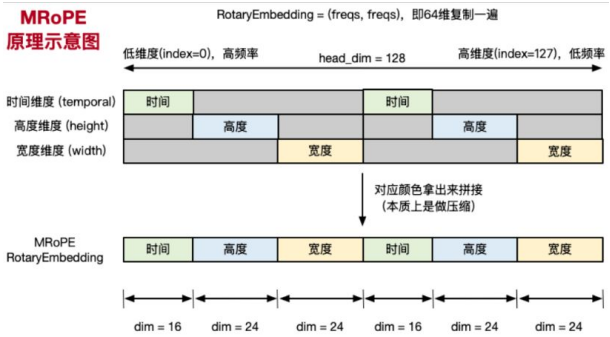


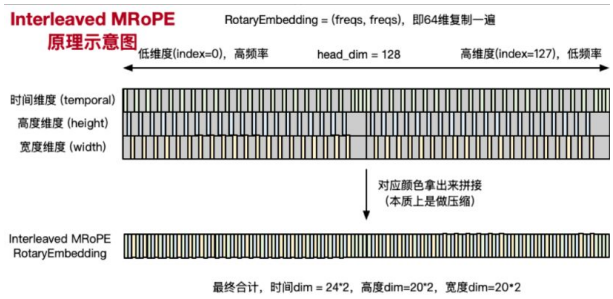
此外`deepstack_visual_indexes`取空列表，这个是 Qwen3-VL 用 DeepStack 把 ViT 若干中间层的特征分别注入 LLM 的前几层，这里**退回到只取最后一层、单点注入**，即不启用 DeepStack。

> 拓展阅读：PatchMerger 是如何合并 2x2 的 Patch 的?

1.5.3 其他配置细节

- **Interleaved MRoPE**：主干的 RoPE 在多模态下是 M-RoPE，把位置拆成时间、高、宽三个通道交错，具体参考 字节面试官：Interleaved MRoPE 相比 MRoPE 改进？。





- **图像 token 不进 n-gram 表**：`ple_input_ids` 是在视觉特征写入之前取的，51B 的 n-gram table 对图像内容不提供任何检索能力，它只对文本 token 起作用：

1.6 MoE

MoE 设置 512 个路由专家、每 token 激活 10 个（Top=10），带门控共享专家 1 个：

1. **激活门控是 softmax 而非 sigmoid，并且没有 loss-free balance 的偏置项。** 这一项主要对比的是同期 GLM-5.3-Flash、DeepSeek V4 的 sigmoid 门控 + 无辅助损失偏置。
2. **共享专家先经过一个逐 token 的标量 sigmoid 门再相加。**`shared_expert_gate` 是`Linear(hidden_size, 1)`，由每个 token 自行决定共享专家输出的权重。

2. Muon 优化器

> 拓展阅读：Muon 优化器

使用 Muon 的权重：注意力的 q/k/v 与输出投影、GDN 的输入输出投影、路由专家与共享专家的 fc1/fc2、n-gram 嵌入层的 key/value 投影。

使用 AdamW 上的权重：输入嵌入、输出头、MoE 路由器、GR 的低秩投影，n-gram 嵌入表使用关闭 weight decay 的 Adam。

- 路由器保留 AdamW 的理由：Muon 加剧早期训练波动并使路由器失稳，即便中后期再启用也没有显著收益。
- GR 的低秩投影同样更适合 AdamW，原因在于其形状极度细长。
- 融合参数必须先切分再正交化：Megatron-LM 里 qkv 投影、SwiGLU 的 fc1、GDN 输入投影都按输出维融合成一个矩阵，语义上却是若干独立线性算子的拼接。直接正交化会把互不相关子块的奇异方向混合。因此在正交化前先切分梯度缓冲、逐块执行 NS 迭代、再聚回原 tensor 布局。

DeepSeek V4 在使用 Muon 优化器上同样选择隔离部分参数使用 AdamW：

模块	优化器
Embedding	AdamW
Prediction head	AdamW
mHC 的静态 bias S_l、gating factor α_l	AdamW
RMSNorm	AdamW
其他所有参数（attention, MoE expert, mHC 等）	**Muon**

- embedding / head 稀疏更新（每次只更新 batch 里出现的 token 行），走 AdamW。
- RMSNorm、mHC 是 1D 向量，Muon 对标量/向量没有意义。

3. 实验

	Qwen3.8-Flash-Next-Base	Qwen3.8-27B-Base	Qwen3.7-Plus-Base
# Params	125B	27B	397B
# Activated Params	6B	27B	17B
# N-gram Embedding Params	51B	–	–
General Tasks			
MMLU	90.36	87.51	**90.43**
MMLU-Redux	90.68	87.26	**91.47**
MMLU-Pro	**73.23**	68.60	70.90
SuperGPQA	**51.36**	44.86	48.42
BBH	**90.87**	89.56	89.41
Math & STEM Tasks			
GPQA	51.42	45.01	**51.52**
GSM8K	**93.29**	93.18	92.95
MATH	72.78	60.54	**74.38**
Coding Tasks			
EvalPlus	**78.76**	76.05	78.06
MultiPL-E	79.09	74.50	**81.68**
SWEBench-Pretrain	**50.99**	41.66	49.24
Multilingual Tasks			
MGSM	**89.33**	86.37	85.42
MMMLU	**84.86**	79.74	84.53
INCLUDE	78.40	74.37	**78.90**

对比上一代 397B-A17B，激活参数只有三分之一，Qwen3.8-Flash-Next 优势主要体现在效率而非绝对能力。因此 Qwen3.8-Flash-Next 更像是 Qwen-4 系列的架构 Scaling 探索而不是完整 Scaling 训练后的产物。对比同期模型，Qwen3.8-Flash-Next 和 Kimi K3、GLM-5.3-Flash、DeepSeek V4 有很多架构设计上的趋同，下篇拆解。